

OpenCV — Detailed Notes

Classical computer vision in Python: read, transform, and analyse images & video — with runnable code, a quick reference, and three complete projects.

1. What OpenCV is

OpenCV (cv2) is the standard Python library for classical computer vision — reading, transforming, and analysing images and video at the pixel level. Images are just NumPy arrays, so everything you know about NumPy applies to them.

Key quirk: OpenCV loads images in BGR order, not RGB, and Hue in HSV ranges 0–179 (not 0–360). Remember both whenever colours look wrong.

2. Installation & how to run

Install with pip:

```
pip install opencv-python          # core library
pip install opencv-contrib-python # extra modules (SIFT, etc.) - optional
```

Write your code in a .py file and run it from the terminal:

```
python my_script.py
```

Important about cv2.imshow: it opens a desktop GUI window — it works on your local machine but not in Google Colab / Jupyter on a server. In Colab use cv2_imshow from google.colab.patches (see Project 5.4), or display with Matplotlib.

Always check the image loaded: cv2.imread returns None for a bad path — guard with `if img is None: ....`

3. Core operations

3.1 Read, show, save an image

```
import cv2

img = cv2.imread("photo.jpg")    # BGR NumPy array (None if path is wrong)
print(img.shape)                 # (height, width, 3)

cv2.imshow("Window", img)        # open a window
cv2.waitKey(0)                   # wait for any key (0 = forever)
cv2.destroyAllWindows()

cv2.imwrite("copy.jpg", img)      # save to disk
```

3.2 Pixels & regions of interest (ROI)

```
px = img[100, 50]                # pixel at row=100, col=50 -> [B, G, R]
img[100, 50] = [0, 0, 255]        # set that pixel to red

roi = img[50:150, 80:200]         # a sub-image (a 'region of interest')
img[0:100, 0:120] = roi           # paste it into another spot
```

Note: arrays are indexed [y, x], but drawing functions take coordinates as (x, y).

3.3 Make a blank canvas

```
import numpy as np
canvas = np.zeros((400, 400, 3), dtype="uint8") # black image
canvas[:] = (255, 255, 255)                    # fill white
```

3.4 Colour conversions

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # to grayscale
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)   # BGR -> RGB (for Matplotlib)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)   # for colour detection
```

3.5 Resize, crop, rotate

```
resized = cv2.resize(img, (300, 200))        # (width, height)
cropped = img[50:200, 100:300]                # slicing: [y1:y2, x1:x2]
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

3.6 Draw shapes & text

```
cv2.rectangle(img, (50, 50), (200, 200), (0, 255, 0), 2) # colour is (B,G,R)
cv2.circle(img, (150, 150), 40, (255, 0, 0), 3)
cv2.line(img, (0, 0), (300, 300), (0, 0, 255), 2)
_  
_ = cv2.putText(img, "Hi", (60, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
```

3.7 Image arithmetic & blending

```
added = cv2.add(img1, img2)                  # saturated addition
blended = cv2.addWeighted(img1, 0.7, img2, 0.3, 0) # 70% + 30% overlay
```

3.8 Bitwise operations & masking

```
mask = np.zeros(img.shape[:2], dtype="uint8")
cv2.circle(mask, (150, 150), 100, 255, -1) # white filled circle
masked = cv2.bitwise_and(img, img, mask=mask) # keep only that region
```

3.9 Blurring (Gaussian, median, bilateral)

```
gauss = cv2.GaussianBlur(img, (7, 7), 0) # general smoothing (odd kernel)
median = cv2.medianBlur(img, 5)           # best for salt-and-pepper noise
bilateral = cv2.bilateralFilter(img, 9, 75, 75) # blurs but keeps edges sharp
```

3.10 Thresholding (simple, adaptive, Otsu)

```
ret, simple = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
adaptive = cv2.adaptiveThreshold(gray, 255,
                                cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
ret, otsu = cv2.threshold(gray, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU) # auto threshold
```

3.11 Morphological operations

```
kernel = np.ones((5, 5), np.uint8)
erode = cv2.erode(thresh, kernel, iterations=1) # shrink white regions
dilate = cv2.dilate(thresh, kernel, iterations=1) # grow white regions
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel) # remove specks
closing = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel) # fill holes
```

3.12 Edges & gradients (Canny, Sobel, Laplacian)

```
edges = cv2.Canny(img, 100, 200) # low, high thresholds
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3) # vertical edges
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3) # horizontal edges
lap = cv2.Laplacian(gray, cv2.CV_64F)
```

3.13 Contours & their properties

```
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2) # -1 = draw all

for c in contours:
    area = cv2.contourArea(c)
    peri = cv2.arcLength(c, True)
    x, y, w, h = cv2.boundingRect(c) # bounding box
    approx = cv2.approxPolyDP(c, 0.02 * peri, True) # simplify to corners
```

3.14 Colour detection in HSV (inRange)

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
lower = np.array([100, 150, 50]) # lower HSV bound (example: blue)
upper = np.array([140, 255, 255]) # upper HSV bound
mask = cv2.inRange(hsv, lower, upper) # white where colour matches
result = cv2.bitwise_and(frame, frame, mask=mask)
```

3.15 Histograms & contrast

```
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
equalized = cv2.equalizeHist(gray) # spreads out contrast
```

3.16 Template matching (find a patch in an image)

```
res = cv2.matchTemplate(img_gray, template_gray, cv2.TM_CCOEFF_NORMED)
min_v, max_v, min_loc, max_loc = cv2.minMaxLoc(res)
top_left = max_loc # best-match location
```

3.17 Perspective transform (top-down warp)

```
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])
M = cv2.getPerspectiveTransform(pts1, pts2)
warped = cv2.warpPerspective(img, M, (300, 300))
```

3.18 Webcam / video loop

```
cap = cv2.VideoCapture(0) # 0 = default camera
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Camera", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"): # press q to quit",
        break
cap.release()
cv2.destroyAllWindows()
```

3.19 Saving a video

```
fourcc = cv2.VideoWriter_fourcc(*"XVID")
out = cv2.VideoWriter("output.avi", fourcc, 20.0, (640, 480))
out.write(frame) # call inside the loop
```

```
out.release()      # call after the loop
```

4. Quick function reference

Function	What it does
cv2.imread / imwrite	Read an image from / write an image to disk.
cv2.imshow / waitKey	Show an image in a window; wait for a key press.
cv2.cvtColor	Convert between colour spaces (BGR, RGB, GRAY, HSV).
cv2.resize	Resize an image to a (width, height).
cv2.GaussianBlur	Smooth / blur an image.
cv2.Canny	Detect edges between two thresholds.
cv2.threshold	Convert grayscale to black/white.
cv2.inRange	Build a mask of pixels inside an HSV colour range.
cv2.findContours	Find outlines of shapes in a binary image.
cv2.boundingRect	Get the (x, y, w, h) box around a contour.
cv2.rectangle / circle / putText	Draw shapes and text onto an image.
cv2.bitwise_and	Combine images / apply a mask.
cv2.CascadeClassifier	Load a Haar cascade and detect objects/faces.
cv2.VideoCapture	Open a webcam or video file for frames.
cv2.warpPerspective	Apply a top-down (bird's-eye) transform.

5. Projects

5.1 Real-Time Face Detection

Opens your webcam and boxes every face in real time using a Haar cascade that ships with OpenCV (nothing to download or train).

```
import cv2

face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml")

cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
                                          minNeighbors=5, minSize=(60, 60))

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, "Face", (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(frame, f"Faces: {len(faces)}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
    cv2.imshow("Face Detection - press q to quit", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
```

Run: save as `face_detect.py`, then `python face_detect.py`. Press q to quit.

5.2 Colour Object Tracker (HSV)

Tracks a coloured object live by masking its HSV range, cleaning the mask, and boxing the largest blob. Tune the lower/upper values for your object's colour.

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
lower = np.array([100, 150, 50]) # example range targets blue
upper = np.array([140, 255, 255])

while True:
    ret, frame = cap.read()
    if not ret:
        break
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, lower, upper)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5,5), np.uint8))

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        c = max(contours, key=cv2.contourArea)
        if cv2.contourArea(c) > 500: # ignore tiny noise
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imshow("Colour Tracker - press q to quit", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
```

Tip: to find HSV values for your object, convert a sample pixel with `cv2.cvtColor` and widen the range a little around it.

5.3 Document Scanner (edges → contour → warp)

Finds a sheet of paper in a photo and flattens it to a clean top-down scan — the same trick scanner apps use.

```
import cv2
import numpy as np

img = cv2.imread("document.jpg")
orig = img.copy()

# 1. grayscale -> blur -> edges
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 75, 200)

# 2. keep the largest 4-corner contour (the page)
cnts, _ = cv2.findContours(edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
cnts = sorted(cnts, key=cv2.contourArea, reverse=True)[:5]
doc = None
for c in cnts:
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)
```

```

    if len(approx) == 4:
        doc = approx
        break

# 3. order corners and warp to a top-down view
def order_points(pts):
    pts = pts.reshape(4, 2)
    rect = np.zeros((4, 2), dtype="float32")
    s = pts.sum(axis=1)
    rect[0] = pts[np.argmin(s)]; rect[2] = pts[np.argmax(s)]
    d = np.diff(pts, axis=1)
    rect[1] = pts[np.argmin(d)]; rect[3] = pts[np.argmax(d)]
    return rect

if doc is not None:
    rect = order_points(doc)
    (tl, tr, br, bl) = rect
    W = int(max(np.linalg.norm(br-bl), np.linalg.norm(tr-tl)))
    H = int(max(np.linalg.norm(tr-br), np.linalg.norm(tl-bl)))
    dst = np.float32([[0,0], [W,0], [W,H], [0,H]])
    M = cv2.getPerspectiveTransform(rect, dst)
    scan = cv2.warpPerspective(orig, M, (W, H))
    cv2.imwrite("scanned.jpg", scan)
else:
    print("No document outline found - try a clearer photo.")

```

5.4 Colab-friendly version (runs on an image, no webcam)

Server notebooks have no camera or GUI. This version reads an image and uses `cv2_imshow` instead of `cv2.imshow`.

```

import cv2
from google.colab.patches import cv2_imshow # Colab display

img = cv2.imread("people.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
faces = face_cascade.detectMultiScale(gray, 1.1, 5)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
print("Faces found:", len(faces))
cv2_imshow(img) # use this in Colab, NOT cv2.imshow

```

6. Common pitfalls & tips

- `imread` returns `None` for a wrong path — check if `img` is `None` before using it.
- Colours look wrong in Matplotlib? Convert BGR → RGB first; OpenCV is BGR.
- Window frozen or not closing? You forgot `waitKey` / `destroyAllWindows`. In a video loop use `waitKey(1)`.
- Blur/morphology kernels should use odd sizes like (3,3), (5,5), (7,7).
- Drawing takes (x, y) but arrays index [y, x] — a frequent source of bugs.
- In HSV, OpenCV's Hue is 0–179 (half the usual 0–360).
- Detection slow on big frames? Resize the frame down before processing.